

One last example should suffice to show how these formulas are derived in general. Suppose the upper limit of integration is infinite and the integrand falls off exponentially. Then we want a change of variable that maps $e^{-x}dx$ into $(\pm)dt$ (with the sign chosen to keep the upper limit of the new variable larger than the lower limit). Doing the integration gives by inspection

$$t = e^{-x} \quad \text{or} \quad x = -\log t \quad (4.4.7)$$

so that

$$\int_{x=a}^{x=\infty} f(x)dx = \int_{t=0}^{t=e^{-a}} f(-\log t) \frac{dt}{t} \quad (4.4.8)$$

The user-transparent implementation would be

```
quadrature.h  template <class T>
               struct Midexp : Midpnt<T>{
This routine is an exact replacement for midpnt, except that bb is assumed to be infinite (value
passed not actually used). It is assumed that the function funk decreases exponentially rapidly
at infinity.
    Doub func(const Doub x) {
        return Midpnt<T>::funkt(-log(x))/x;          Effect the change of variable.
    }
    Midexp(T &funcc, const Doub aa, const Doub bb) :
        Midpnt<T>(funcc, aa, bb) {
        Midpnt<T>::a=0.0;
        Midpnt<T>::b=exp(-aa);
    }
};
```

CITED REFERENCES AND FURTHER READING:

Acton, F.S. 1970, *Numerical Methods That Work*; 1990, corrected edition (Washington, DC: Mathematical Association of America), Chapter 4.

Dahlquist, G., and Bjorck, A. 1974, *Numerical Methods* (Englewood Cliffs, NJ: Prentice-Hall); reprinted 2003 (New York: Dover), §7.4.3, p. 294.

Stoer, J., and Bulirsch, R. 2002, *Introduction to Numerical Analysis*, 3rd ed. (New York: Springer), §3.7.

4.5 Quadrature by Variable Transformation

Imagine a simple general quadrature algorithm that is very rapidly convergent and allows you to ignore endpoint singularities completely. Sound too good to be true? In this section we'll describe an algorithm that in fact handles large classes of integrals in exactly this way.

Consider evaluating the integral

$$I = \int_a^b f(x)dx \quad (4.5.1)$$

As we saw in the construction of equations (4.1.11) – (4.1.20), quadrature formulas of arbitrarily high order can be constructed with interior weights unity, just by tuning the weights near the endpoints. But if a function dies off rapidly enough near

the endpoints, then those weights don't matter at all. In such a case, an N -point quadrature with uniform weights converges exponentially with N . (For a more rigorous motivation of this idea, see §4.5.1. For the connection to Gaussian quadrature, see the discussion at the end of §20.7.4.)

What about a function that doesn't vanish at the endpoints? Consider a change of variables $x = x(t)$, such that $x \in [a, b] \rightarrow t \in [c, d]$:

$$I = \int_c^d f[x(t)] \frac{dx}{dt} dt \quad (4.5.2)$$

Choose the transformation such that the factor dx/dt goes rapidly to zero at the endpoints of the interval. Then the simple trapezoidal rule applied to (4.5.2) will give extremely accurate results. (In this section, we'll call quadrature with uniform weights trapezoidal quadrature, with the understanding that it's a matter of taste whether you weight the endpoints with weight $1/2$ or 1 , since they don't count anyway.)

Even when $f(x)$ has integrable singularities at the endpoints of the interval, their effect can be overwhelmed by a suitable transformation $x = x(t)$. One need not tailor the transformation to the specific nature of the singularity: We will discuss several transformations that are effective at obliterating just about any kind of endpoint singularity.

The first transformation of this kind was introduced by Schwartz [1] and has become known as the TANH rule:

$$\begin{aligned} x &= \frac{1}{2}(b+a) + \frac{1}{2}(b-a) \tanh t, & x \in [a, b] \rightarrow t \in [-\infty, \infty] \\ \frac{dx}{dt} &= \frac{1}{2}(b-a) \operatorname{sech}^2 t = \frac{2}{b-a}(b-x)(x-a) \end{aligned} \quad (4.5.3)$$

The sharp decrease of $\operatorname{sech}^2 t$ as $t \rightarrow \pm\infty$ explains the efficiency of the algorithm and its ability to deal with singularities. Another similar algorithm is the IMT rule [2]. However, $x(t)$ for the IMT rule is not given by a simple analytic expression, and its performance is not too different from the TANH rule.

There are two kinds of errors to consider when using something like the TANH rule. The *discretization error* is just the truncation error because you are using the trapezoidal rule to approximate I . The *trimming error* is the result of truncating the infinite sum in the trapezoidal rule at a finite value of N . (Recall that the limits are now $\pm\infty$.) You might think that the sharper the decrease of dx/dt as $t \rightarrow \pm\infty$, the more efficient the algorithm. But if the decrease is too sharp, then the density of quadrature points near the center of the original interval $[a, b]$ is low and the discretization error is large. The optimal strategy is to try to arrange that the discretization and trimming errors are approximately equal.

For the TANH rule, Schwartz [1] showed that the discretization error is of order

$$\epsilon_d \sim e^{-2\pi w/h} \quad (4.5.4)$$

where w is the distance from the real axis to the nearest singularity of the integrand. There is a pole when $\operatorname{sech}^2 t \rightarrow \infty$, i.e., when $t = \pm i\pi/2$. If there are no poles closer to the real axis in $f(x)$, then $w = \pi/2$. The trimming error, on the other hand, is

$$\epsilon_t \sim \operatorname{sech}^2 t_N \sim e^{-2Nh} \quad (4.5.5)$$

Setting $\epsilon_d \sim \epsilon_t$, we find

$$h \sim \frac{\pi}{(2N)^{1/2}}, \quad \epsilon \sim e^{-\pi(2N)^{1/2}} \quad (4.5.6)$$

as the optimum h and the corresponding error. Note that ϵ decreases with N faster than any power of N . If f is singular at the endpoints, this can modify equation (4.5.5) for ϵ_t . This usually results in the constant π in (4.5.6) being reduced. Rather than developing an algorithm where we try to estimate the optimal h for each integrand a priori, we recommend simple step doubling and testing for convergence. We expect convergence to set in for h around the value given by equation (4.5.6).

The TANH rule essentially uses an exponential mapping to achieve the desired rapid fall-off at infinity. On the theory that more is better, one can try repeating the procedure. This leads to the DE (double exponential) rule:

$$\begin{aligned} x &= \frac{1}{2}(b+a) + \frac{1}{2}(b-a) \tanh(c \sinh t), & x \in [a, b] \rightarrow t \in [-\infty, \infty] \\ \frac{dx}{dt} &= \frac{1}{2}(b-a) \operatorname{sech}^2(c \sinh t) c \cosh t \sim \exp(-c \exp |t|) \quad \text{as } |t| \rightarrow \infty \end{aligned} \quad (4.5.7)$$

Here the constant c is usually taken to be 1 or $\pi/2$. (Values larger than $\pi/2$ are not useful since $w = \pi/2$ for $0 < c \leq \pi/2$, but w decreases rapidly for larger c .) By an analysis similar to equations (4.5.4) – (4.5.6), one can show that the optimal h and corresponding error for the DE rule are of order

$$h \sim \frac{\log(2\pi N w/c)}{N}, \quad \epsilon \sim e^{-kN/\log N} \quad (4.5.8)$$

where k is a constant. The improved performance of the DE rule over the TANH rule indicated by comparing equations (4.5.6) and (4.5.8) is borne out in practice.

4.5.1 Exponential Convergence of the Trapezoidal Rule

The error in evaluating the integral (4.5.1) by the trapezoidal rule is given by the Euler-Maclaurin summation formula,

$$I \approx \frac{h}{2}[f(a) + f(b)] + h \sum_{j=1}^{N-1} f(a+jh) - \sum_{k=1}^{\infty} \frac{B_{2k} h^{2k}}{(2k)!} [f^{(2k-1)}(b) - f^{(2k-1)}(a)] \quad (4.5.9)$$

Note that this is in general an asymptotic expansion, not a convergent series. If all the derivatives of the function f vanish at the endpoints, then all the “correction terms” in equation (4.5.9) are zero. The error in this case is very small — it goes to zero with h faster than any power of h . We say that the method converges *exponentially*. The straight trapezoidal rule is thus an excellent method for integrating functions such as $\exp(-x^2)$ on $(-\infty, \infty)$, whose derivatives all vanish at the endpoints.

The class of transformations that will produce exponential convergence for a function whose derivatives do not all vanish at the endpoints is those for which dx/dt and all its derivatives go to zero at the endpoints of the interval. For functions with singularities at the endpoints, we require that $f(x) dx/dt$ and all its derivatives vanish at the endpoints. This is a more precise statement of “ dx/dt goes rapidly to zero” given above.

4.5.2 Implementation

Implementing the DE rule is a little tricky. It's not a good idea to simply use `Trapzd` on the function $f(x) dx/dt$. First, the factor $\text{sech}^2(c \sinh t)$ in equation (4.5.7) can overflow if `sech` is computed as $1/\cosh$. We follow [3] and avoid this by using the variable q defined by

$$q = e^{-2 \sinh t} \quad (4.5.10)$$

(we take $c = 1$ for simplicity) so that

$$\frac{dx}{dt} = 2(b-a) \frac{q}{(1+q)^2} \cosh t \quad (4.5.11)$$

For large positive t , q just underflows harmlessly to zero. Negative t is handled by using the symmetry of the trapezoidal rule about the midpoint of the interval. We write

$$\begin{aligned} I &\simeq h \sum_{j=-N}^N f(x_j) \left. \frac{dx}{dt} \right|_j \\ &= h \left\{ f[(a+b)/2] \left. \frac{dx}{dt} \right|_0 + \sum_{j=1}^N [f(a+\delta_j) + f(b-\delta_j)] \left. \frac{dx}{dt} \right|_j \right\} \end{aligned} \quad (4.5.12)$$

where

$$\delta = b - x = (b-a) \frac{q}{1+q} \quad (4.5.13)$$

A second possible problem is that cancellation errors in computing $a+\delta$ or $b-\delta$ can cause the computed value of $f(x)$ to blow up near the endpoint singularities. To handle this, you should code the function $f(x)$ as a function of two arguments, $f(x, \delta)$. Then compute the singular part using δ directly. For example, code the function $x^{-\alpha}(1-x)^{-\beta}$ as $\delta^{-\alpha}(1-x)^{-\beta}$ near $x = 0$ and $x^{-\alpha}\delta^{-\beta}$ near $x = 1$. (See §6.10 for another example of a $f(x, \delta)$.) Accordingly, the routine `DERule` below expects the function f to have two arguments. If your function has no singularities, or the singularities are “mild” (e.g., no worse than logarithmic), you can ignore δ when coding $f(x, \delta)$ and code it as if it were just $f(x)$.

The routine `DERule` implements equation (4.5.12). It contains an argument $h_{\max}$ that corresponds to the upper limit for t . The first approximation to I is given by the first term on the right-hand side of (4.5.12) with $h = h_{\max}$. Subsequent refinements correspond to halving h as usual. We typically take $h_{\max} = 3.7$ in double precision, corresponding to $q = 3 \times 10^{-18}$. This is generally adequate for “mild” singularities, like logarithms. If you want high accuracy for stronger singularities, you may have to increase $h_{\max}$. For example, for $1/\sqrt{x}$ you need $h_{\max} = 4.3$ to get full double precision. This corresponds to $q = 10^{-32} = (10^{-16})^2$, as you might expect.

```
template<class T>
struct DERule : Quadrature {
    Structure for implementing the DE rule.
    Doub a,b,hmax,s;
    T &func;
```

`derule.h`

```
DERule(T &funcc, const Doub aa, const Doub bb, const Doub hmaxx=3.7)
: func(funcc), a(aa), b(bb), hmax(hmaxx) {n=0;}
```

Constructor. `funcc` is the function or functor that provides the function to be integrated between limits `aa` and `bb`, also input. The function operator in `funcc` takes two arguments, x and δ , as described in the text. The range of integration in the transformed variable t is $(-hmaxx, hmaxx)$. Typical values of `hmaxx` are 3.7 for logarithmic or milder singularities, and 4.3 for square-root singularities, as discussed in the text.

```
Doub next() {
On the first call to the function next ( $n = 1$ ), the routine returns the crudest estimate of
 $\int_a^b f(x)dx$ . Subsequent calls to next ( $n = 2, 3, \dots$ ) will improve the accuracy by adding
 $2^{n-1}$  additional interior points.
Doub del,fact,q,sum,t,twoh;
Int it,j;
n++;
if (n == 1) {
fact=0.25;
return s=hmax*2.0*(b-a)*fact*func(0.5*(b+a),0.5*(b-a));
} else {
for (it=1,j=1;j<n-1;j++) it <= 1;
twoh=hmax/it;
t=0.5*twoh;
for (sum=0.0,j=0;j<it;j++) {
q=exp(-2.0*sinh(t));
del=(b-a)*q/(1.0+q);
fact=q/SQR(1.0+q)*cosh(t);
sum += fact*(func(a+del,del)+func(b-del,del));
t += twoh;
}
return s=0.5*s+(b-a)*twoh*sum; Replace s by its refined value and return.
}
};
```

If the double exponential rule (DE rule) is generally better than the single exponential rule (TANH rule), why don't we keep going and use a triple exponential rule, quadruple exponential rule, ...? As we mentioned earlier, the discretization error is dominated by the pole nearest to the real axis. It turns out that beyond the double exponential the poles come nearer and nearer to the real axis, so the methods tend to get worse, not better.

If the function to be integrated itself has a pole near the real axis (much nearer than the $\pi/2$ that comes from the DE or TANH rules), the convergence of the method slows down. In analytically tractable cases, one can find a "pole correction term" to add to the trapezoidal rule to restore rapid convergence [4].

4.5.3 Infinite Ranges

Simple variations of the TANH or DE rules can be used if either or both of the limits of integration is infinite:

Range	TANH Rule	DE Rule	Mixed Rule	(4.5.14)
$(0, \infty)$	$x = e^t$	$x = e^{2c \sinh t}$	$x = e^{t-e^{-t}}$	
$(-\infty, \infty)$	$x = \sinh t$	$x = \sinh(c \sinh t)$	—	

The last column gives a mixed rule for functions that fall off rapidly (e^{-x} or e^{-x^2}) at infinity. It is a DE rule at $x = 0$ but only a single exponential at infinity. The expo-